

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО»
(Университет ИТМО)

Факультет **Прикладной информатики**

Образовательная программа **Мобильные и облачные технологии**

Направление подготовки **09.03.03 Прикладная информатика**

О Т Ч Е Т

по ЛАБОРАТОРНОЙ РАБОТЕ №5

Тема: **ПРОЦЕДУРЫ, ФУНКЦИИ, ТРИГГЕРЫ В PostgreSQL**

Выполнили: Сатрио Субагио K3240

Преподаватель : Говорова Марина Михайловна

Дата 06.05.2026

Санкт-Петербург 2026

1 Цель работы

Овладеть практическими навыками создания и использования процедур, функций и триггеров в базе данных PostgreSQL. Освоить технологии автоматизации бизнес-логики с помощью хранимых процедур и обеспечения целостности данных и аудита изменений с помощью триггеров.

2 Практическое задание

Создать 3 хранимые процедуры для индивидуальной базы данных (labcompany_database) согласно заданию части 4 ЛР 2:

1. Процедура повышения оклада сотрудникам, выполнившим задания с опережением графика.
2. Процедура вычисления количества проектов, в которых участвует сотрудник.
3. Процедура поиска номера телефона отдела сотрудника.

Создать триггеры для индивидуальной БД (Вариант 2.1):

1. Разработать систему аудита (логирования) действий.
2. Реализовать 10 триггеров для таблиц employee, project, task, и contract для фиксации операций INSERT, UPDATE и DELETE в таблице audit_logs.

3 Выполнение работы

3.1 Хранимые процедуры

Были разработаны три процедуры на языке PL/pgSQL для управления данными компании.

Процедура повышения оклада сотрудникам, выполнившим задания с опережением графика.

Процедура 1: Повышение зарплаты (Salary Increase)

1. Процедура принимает входной параметр `p_increase_percent` (числовое значение процента повышения).
2. Система идентифицирует всех активных сотрудников (у которых `end_date` IS NULL в таблице истории HR), которые имеют хотя бы одну задачу со статусом 'Completed' в таблице `task`. Связь осуществляется через таблицы `participation` и `hr_history`.
3. Для позиций (`position`), занимаемых этими сотрудниками, выполняется обновление поля `salary`.
4. Новая зарплата рассчитывается по формуле: Новая ЗП = Текущая ЗП * (1 + процент / 100).
5. После успешного обновления выводится уведомление о количестве повышенных окладов.

```
labcompany_database=# -- Procedure 1: Increase salary for employees with completed tasks
labcompany_database=# -- Input: p_increase_percent (e.g., 10 for 10%)
labcompany_database=# CREATE OR REPLACE PROCEDURE company_schema2.increase_salary_early_finishers(
labcompany_database=#     IN p_increase_percent NUMERIC
labcompany_database=# )
labcompany_database=# LANGUAGE plpgsql
labcompany_database=# AS $$
labcompany_database$# BEGIN
labcompany_database$#     -- Update salary in the 'position' table
labcompany_database$#     -- Only for positions held by employees who have a 'Completed' task
labcompany_database$#     UPDATE company_schema2."position" pos
labcompany_database$#     SET salary = salary * (1.0 + p_increase_percent / 100.0)
labcompany_database$#     WHERE pos.id_position IN (
labcompany_database$#         SELECT DISTINCT hr.id_position
labcompany_database$#         FROM company_schema2.hr_history hr
labcompany_database$#         JOIN company_schema2.participation part ON hr.id_record = part.id_hr_record
labcompany_database$#         JOIN company_schema2.task t ON part.id_task = t.id_task
labcompany_database$#         WHERE t.status_name = 'Completed'
labcompany_database$#         AND hr.end_date IS NULL -- Check only active employees
labcompany_database$#     );
labcompany_database$#     -- Show a success message
labcompany_database$#     RAISE NOTICE 'Salary increased by % percent for employees with completed tasks.', p_increase_p
labcompany_database$#     ercent;
labcompany_database$# END;
labcompany_database$# $$;
CREATE PROCEDURE
```

Рисунок 1 — Хранимые процедуры - Повышение зарплаты

```
labcompany_database=# CALL company_schema2.increase_salary_early_finishers(10);
NOTICE:  Salary increased by 10 percent for employees with completed tasks.
CALL
```

Рисунок 2 — команда "call"- Повышение зарплаты

Процедура вычисления количества проектов, в которых участвует сотрудник.

Процедура 2: Подсчет проектов сотрудника (Count Projects)

1. Процедура принимает входной параметр `p_employee_id` (ID сотрудника) и выходной параметр `p_project_count` (для возврата результата).
2. Система выполняет соединение таблиц `project`, `task`, `participation` и `hr_history` для определения всех проектов, в которых участвует указанный сотрудник.
3. Выбираются только активные записи сотрудника (`end_date IS NULL`).
4. С помощью функции `COUNT(DISTINCT ...)` подсчитывается количество уникальных проектов.
5. Если проектов не найдено (результат `NULL`), счетчик устанавливается в 0, и выводится итоговое сообщение.

```
labcompany_database=# -- Procedure 2: Get total project count for an employee
labcompany_database=# -- Input: p_employee_id (ID of the employee)
labcompany_database=# -- Output: p_project_count (Total number of projects)
labcompany_database=# CREATE OR REPLACE PROCEDURE company_schema2.get_employee_project_count(
labcompany_database=#     IN p_employee_id INTEGER,
labcompany_database=#     INOUT p_project_count INTEGER
labcompany_database=# )
labcompany_database=# LANGUAGE plpgsql
labcompany_database=# AS $$
labcompany_database=# BEGIN
labcompany_database=#     -- Count distinct projects linked to the employee
labcompany_database=#     SELECT COUNT(DISTINCT p.id_project)
labcompany_database=#     INTO p_project_count
labcompany_database=#     FROM company_schema2.project p
labcompany_database=#     JOIN company_schema2.task t ON p.id_project = t.id_project
labcompany_database=#     JOIN company_schema2.participation part ON t.id_task = part.id_task
labcompany_database=#     JOIN company_schema2.hr_history hr ON part.id_hr_record = hr.id_record
labcompany_database=#     WHERE hr.id_employee = p_employee_id
labcompany_database=#           AND hr.end_date IS NULL; -- Check only active employees
labcompany_database=#
labcompany_database=#     -- If no projects found, set count to 0 instead of NULL
labcompany_database=#     IF p_project_count IS NULL THEN
labcompany_database=#         p_project_count := 0;
labcompany_database=#     END IF;
labcompany_database=#
labcompany_database=#     -- Show the result message
labcompany_database=#     RAISE NOTICE 'Employee ID % participates in % project(s).', p_employee_id, p_project_count;
labcompany_database=# END;
labcompany_database=# $$;
CREATE PROCEDURE
```

Рисунок 3 — Хранимые процедуры - Подсчет проектов сотрудника

```
labcompany_database=# DO $$
labcompany_database=# DECLARE
labcompany_database=#     v_count INTEGER;
labcompany_database=# BEGIN
labcompany_database=#     CALL company_schema2.get_employee_project_count(1, v_count);
labcompany_database=#     RAISE NOTICE 'Final Result: Employee 1 is in % projects.', v_count;
labcompany_database=# END $$;
NOTICE: Employee ID 1 participates in 2 project(s).
NOTICE: Final Result: Employee 1 is in 2 projects.
DO
```

Рисунок 4 — команда "call"

Процедура поиска номера телефона отдела сотрудника.

Процедура 3: Поиск телефона отдела (Find Department Phone)

1. Процедура принимает входной параметр `p_surname` (фамилия сотрудника) и выходной параметр `p_phone_number`.
2. Система ищет сотрудника по фамилии в таблице `employee` и через таблицу `hr_history` определяет его текущий отдел.
3. Учитываются только активные записи (`end_date IS NULL`).
4. Из таблицы `department` извлекается номер телефона (`phone`).
5. Если сотрудник не найден, генерируется исключение (`EXCEPTION`); иначе выводится номер телефона.

```
labcompany_database=# BEGIN
labcompany_database=# -- Get the phone number from the department table
labcompany_database=# SELECT d.phone
labcompany_database=# INTO p_phone_number
labcompany_database=# FROM company_schema2.employee e
labcompany_database=# JOIN company_schema2.hr_history hr ON e.id_employee = hr.id_employee
labcompany_database=# JOIN company_schema2.department d ON hr.id_department = d.id_department
labcompany_database=# WHERE e.surname = p_surname
labcompany_database=# AND hr.end_date IS NULL -- Check only active employees
labcompany_database=# LIMIT 1;
labcompany_database=# -- If no employee found, stop and show an error
labcompany_database=# IF p_phone_number IS NULL THEN
labcompany_database=# RAISE EXCEPTION 'No active employee found with surname "%".', p_surname;
labcompany_database=# END IF;
labcompany_database=# -- Show the result message
labcompany_database=# RAISE NOTICE 'Department phone for % is: %', p_surname, p_phone_number;
labcompany_database=# END;
labcompany_database=# $$;
CREATE PROCEDURE
```

Рисунок 5 — Хранимые процедуры - Подсчет проектов сотрудника

```
labcompany_database=# DO $$
labcompany_database=# DECLARE
labcompany_database=# v_phone VARCHAR;
labcompany_database=# BEGIN
labcompany_database=# CALL company_schema2.get_department_phone_by_employee('Ivanov', v_phone);
labcompany_database=# RAISE NOTICE 'Final Result: Phone number is %', v_phone;
labcompany_database=# END $$;
NOTICE: Department phone for Ivanov is: 021-5550101
NOTICE: Final Result: Phone number is 021-5550101
DO
```

Рисунок 6 — команда "call"

3.2. Триггеры аудита

Для реализации системы аудита была создана таблица логов и универсальная триггерная функция, которая динамически определяет тип операции и идентификатор записи.

```
labcompany_database=# CREATE TABLE IF NOT EXISTS company_schema2.audit_logs (  
labcompany_database(#     log_id SERIAL PRIMARY KEY,  
labcompany_database(#     table_name VARCHAR(50),  
labcompany_database(#     operation VARCHAR(10),  
labcompany_database(#     record_id INTEGER,  
labcompany_database(#     changed_by VARCHAR(50) DEFAULT CURRENT_USER,  
labcompany_database(#     changed_at TIMESTAMP DEFAULT NOW()  
labcompany_database(# );  
CREATE TABLE
```

Рисунок 7 — Создание таблицы логов

Создание триггерной функции

Логика работы триггерной функции аудита (fn_audit_log)

Функция fn_audit_log является универсальным механизмом, который автоматически фиксирует любые изменения данных в таблицах employee, project, task и contract. Алгоритм работы функции следующий:

1. Определение типа операции: При срабатывании триггера функция сначала считывает системную переменную TG_OP, чтобы определить тип выполненного действия: INSERT (добавление), UPDATE (изменение) или DELETE (удаление).
2. Идентификация таблицы и записи: Функция использует переменную TG_TABLE_NAME для определения имени таблицы, в которой произошло событие. В зависимости от имени таблицы (employee, project, task или contract), функция динамически выбирает соответствующее поле первичного ключа (id_employee, id_project, id_task или id_contract) из объекта NEW (для INSERT/UPDATE) или OLD (для DELETE) и сохраняет его в переменную v_record_id.
3. Обработка данных в формате JSON:
4. Для операций INSERT: функция преобразует новую запись (NEW) в текстовый формат JSON и сохраняет её в поле new_data. Поле old_data остается пустым.
5. Для операций UPDATE: функция преобразует старую запись (OLD) в JSON для поля old_data и новую запись (NEW) для поля new_data, фиксируя таким образом полную историю изменений.
6. Для операций DELETE: функция преобразует удаляемую запись (OLD) в JSON для

поля old_data. Поле new_data остается пустым.

7. Фиксация метаданных: Автоматически заполняются служебные поля: имя таблицы (table_name), тип операции (operation), имя пользователя, выполнившего действие (changed_by = CURRENT_USER), и текущее время сервера (changed_at = NOW()).
8. Запись в журнал аудита: Сформированная строка логов вставляется в таблицу company_schema2.audit_logs. После успешной записи функция возвращает объект (NEW или OLD), позволяя исходной операции (вставке, обновлению или удалению) завершиться штатно.

```
labcompany_database=# CREATE OR REPLACE FUNCTION company_schema2.fn_audit_log()
labcompany_database=# RETURNS TRIGGER AS $$
labcompany_database=# DECLARE
labcompany_database=#     v_record_id INTEGER;
labcompany_database=#     v_old_data TEXT;
labcompany_database=#     v_new_data TEXT;
labcompany_database=# BEGIN
labcompany_database=#     IF TG_OP = 'INSERT' THEN
labcompany_database=#         v_new_data := row_to_json(NEW)::text;
labcompany_database=#
labcompany_database=#         IF TG_TABLE_NAME = 'employee' THEN v_record_id := NEW.id_employee;
labcompany_database=#         ELSIF TG_TABLE_NAME = 'project' THEN v_record_id := NEW.id_project;
labcompany_database=#         ELSIF TG_TABLE_NAME = 'task' THEN v_record_id := NEW.id_task;
labcompany_database=#         ELSIF TG_TABLE_NAME = 'contract' THEN v_record_id := NEW.id_contract;
labcompany_database=#         ELSIF TG_TABLE_NAME = 'department' THEN v_record_id := NEW.id_department;
labcompany_database=#         ELSIF TG_TABLE_NAME = 'position' THEN v_record_id := NEW.id_position;
labcompany_database=#         ELSE v_record_id := NULL;
labcompany_database=#         END IF;
labcompany_database=#
labcompany_database=#         INSERT INTO company_schema2.audit_logs (table_name, operation, record_id, new_data)
labcompany_database=#         VALUES (TG_TABLE_NAME, 'INSERT', v_record_id, v_new_data);
labcompany_database=#         RETURN NEW;
labcompany_database=#
labcompany_database=#     ELSIF TG_OP = 'UPDATE' THEN
labcompany_database=#         v_old_data := row_to_json(OLD)::text;
labcompany_database=#         v_new_data := row_to_json(NEW)::text;
labcompany_database=#
labcompany_database=#         IF TG_TABLE_NAME = 'employee' THEN v_record_id := NEW.id_employee;
labcompany_database=#         ELSIF TG_TABLE_NAME = 'project' THEN v_record_id := NEW.id_project;
labcompany_database=#
labcompany_database=#         IF TG_TABLE_NAME = 'employee' THEN v_record_id := NEW.id_employee;
labcompany_database=#         ELSIF TG_TABLE_NAME = 'project' THEN v_record_id := NEW.id_project;
labcompany_database=#         ELSIF TG_TABLE_NAME = 'task' THEN v_record_id := NEW.id_task;
labcompany_database=#         ELSIF TG_TABLE_NAME = 'contract' THEN v_record_id := NEW.id_contract;
labcompany_database=#         ELSIF TG_TABLE_NAME = 'department' THEN v_record_id := NEW.id_department;
labcompany_database=#         ELSIF TG_TABLE_NAME = 'position' THEN v_record_id := NEW.id_position;
labcompany_database=#         ELSE v_record_id := NULL;
labcompany_database=#         END IF;
labcompany_database=#
labcompany_database=#         INSERT INTO company_schema2.audit_logs (table_name, operation, record_id, old_data, new_data)
labcompany_database=#         VALUES (TG_TABLE_NAME, 'UPDATE', v_record_id, v_old_data, v_new_data);
labcompany_database=#         RETURN NEW;
labcompany_database=#
labcompany_database=#     ELSIF TG_OP = 'DELETE' THEN
labcompany_database=#         v_old_data := row_to_json(OLD)::text;
labcompany_database=#
labcompany_database=#         IF TG_TABLE_NAME = 'employee' THEN v_record_id := OLD.id_employee;
labcompany_database=#         ELSIF TG_TABLE_NAME = 'project' THEN v_record_id := OLD.id_project;
labcompany_database=#         ELSIF TG_TABLE_NAME = 'task' THEN v_record_id := OLD.id_task;
labcompany_database=#         ELSIF TG_TABLE_NAME = 'contract' THEN v_record_id := OLD.id_contract;
labcompany_database=#         ELSIF TG_TABLE_NAME = 'department' THEN v_record_id := OLD.id_department;
labcompany_database=#         ELSIF TG_TABLE_NAME = 'position' THEN v_record_id := OLD.id_position;
labcompany_database=#         ELSE v_record_id := NULL;
labcompany_database=#         END IF;
```

Рисунок 8 — Создание триггерной функции

Создание 10 триггеров

```
labcompany_database=# CREATE TRIGGER trg_contract_insert
labcompany_database=# AFTER INSERT ON company_schema2.contract
labcompany_database=# FOR EACH ROW EXECUTE PROCEDURE company_schema2.fn_audit_log();
labcompany_database=# CREATE TRIGGER
```

```
labcompany_database=# CREATE TRIGGER trg_task_delete
labcompany_database=# AFTER DELETE ON company_schema2.task
labcompany_database=# FOR EACH ROW EXECUTE PROCEDURE company_schema2.fn_audit_log();
CREATE TRIGGER
labcompany_database=# |
labcompany_database=# CREATE TRIGGER trg_task_update
labcompany_database=# AFTER UPDATE ON company_schema2.task
labcompany_database=# FOR EACH ROW EXECUTE PROCEDURE company_schema2.fn_audit_log();
CREATE TRIGGER
labcompany_database=# |
```

```
labcompany_database=# CREATE TRIGGER trg_task_insert
labcompany_database=# AFTER INSERT ON company_schema2.task
labcompany_database=# FOR EACH ROW EXECUTE PROCEDURE company_schema2.fn_audit_log();
CREATE TRIGGER
labcompany_database=# |
```

```
labcompany_database=# CREATE TRIGGER trg_project_update
labcompany_database=# AFTER UPDATE ON company_schema2.project
labcompany_database=# FOR EACH ROW EXECUTE PROCEDURE company_schema2.fn_audit_log();
CREATE TRIGGER
labcompany_database=# |
```

```
labcompany_database=# CREATE TRIGGER trg_project_insert
labcompany_database=# AFTER INSERT ON company_schema2.project
labcompany_database=# FOR EACH ROW EXECUTE PROCEDURE company_schema2.fn_audit_log();
CREATE TRIGGER
labcompany_database=# |
```

```
labcompany_database=# CREATE TRIGGER trg_employee_delete
labcompany_database=# AFTER DELETE ON company_schema2.employee
labcompany_database=# FOR EACH ROW EXECUTE PROCEDURE company_schema2.fn_audit_log();
CREATE TRIGGER
labcompany_database=# |
```

```
labcompany_database=# CREATE TRIGGER trg_employee_update
labcompany_database=# AFTER UPDATE ON company_schema2.employee
labcompany_database=# FOR EACH ROW EXECUTE PROCEDURE company_schema2.fn_audit_log();
CREATE TRIGGER
labcompany_database=# |
```

```
labcompany_database=# CREATE TRIGGER trg_employee_insert
labcompany_database=# AFTER INSERT ON company_schema2.employee
labcompany_database=# FOR EACH ROW EXECUTE PROCEDURE company_schema2.fn_audit_log();
CREATE TRIGGER
labcompany_database=# |
```

```
labcompany_database=# CREATE TRIGGER trg_contract_update
labcompany_database=# AFTER UPDATE ON company_schema2.contract
labcompany_database=# FOR EACH ROW EXECUTE PROCEDURE company_schema2.fn_audit_log();
CREATE TRIGGER
labcompany_database=# |
```


Результаты тестирования триггеров

```
labcompany_database=# INSERT INTO company_schema2.contract (number, start_date, status, payment_status, id_organization)
labcompany_database=# VALUES ('CONTRACT-AUDIT-001', CURRENT_DATE, 'Active', 'Unpaid', 1);
INSERT 0 1
labcompany_database=#
```

```
labcompany_database=# UPDATE company_schema2.contract
labcompany_database=# SET payment_status = 'Paid'
labcompany_database=# WHERE number = 'CONTRACT-AUDIT-001';
UPDATE 1
labcompany_database=#
```

```
labcompany_database=# SELECT
labcompany_database=#     log_id,
labcompany_database=#     table_name,
labcompany_database=#     operation,
labcompany_database=#     record_id,
labcompany_database=#     changed_at
labcompany_database=# FROM company_schema2.audit_logs
labcompany_database=# ORDER BY log_id DESC
labcompany_database=# LIMIT 15;
 log_id | table_name | operation | record_id |          changed_at
-----+-----+-----+-----+-----
      14 | contract   | UPDATE    |         11 | 2026-05-07 18:26:23.612647
      13 | contract   | INSERT    |         11 | 2026-05-07 18:26:06.63516
      12 | project    | UPDATE    |          1 | 2026-05-07 18:24:47.309505
      11 | project    | INSERT    |         11 | 2026-05-07 18:24:30.047758
      10 | employee   | DELETE    |         13 | 2026-05-07 18:23:40.747799
       9 | employee   | UPDATE    |          1 | 2026-05-07 18:23:23.822649
       8 | employee   | INSERT    |         13 | 2026-05-07 18:22:55.655365
       7 | task       | DELETE    |         17 | 2026-05-07 18:17:49.471753
       6 | task       | UPDATE    |         17 | 2026-05-07 18:17:36.468808
       5 | task       | INSERT    |         17 | 2026-05-07 18:17:06.009929
       4 | project    | UPDATE    |          2 | 2026-05-07 18:13:50.372702
       3 | project    | UPDATE    |          1 | 2026-05-07 18:13:20.69686
       2 | project    | UPDATE    |          1 | 2026-05-07 17:47:05.168583
       1 | employee   | UPDATE    |          1 | 2026-05-07 17:43:30.45315
(14 rows)
```

ВЫВОДЫ

В ходе выполнения лабораторной работы были успешно освоены навыки работы с хранимыми процедурами и триггерами в PostgreSQL.

Разработаны и протестированы 3 хранимые процедуры, решающие задачи повышения зарплаты, подсчета участия в проектах и поиска контактной информации. Использование процедур позволило инкапсулировать сложную бизнес-логику на стороне сервера.

Реализована система аудита данных с помощью 10 триггеров. Была создана универсальная триггерная функция `fn_audit_log`, которая отслеживает операции `INSERT`, `UPDATE` и `DELETE` в таблицах `employee`, `project`, `task` и `contract`.

Тестирование подтвердило корректность работы системы: любые изменения в основных таблицах

автоматически фиксируются в таблице `audit_logs` с указанием типа операции, идентификатора записи, старых и новых данных, а также времени и автора изменения. Это обеспечивает полную прозрачность и отслеживаемость действий в базе данных.